Power & connectivity — PixelPine rev A

Power input

Item	Value
Connector	USB-C receptacle, 6-pin power-only (VBUS, GND, CC1, CC2), SHOU HAN TYPE-C 6P, SMD
Negotiation	2 × 5.1 kΩ pull-downs on CC1/CC2 (Rd), so any USB-C source treats the tree as a sink and turns VBUS on. Works with C-to-C chargers, laptop ports and USB-A-to-C cables. No USB-PD, no data lines.
Supply voltage	5 V nominal (USB: 4.75–5.25 V). The PY32F003 runs directly from VBUS (rated 1.7–5.5 V), **no regulator**
Protection	None beyond the MCU's own pin diodes and R19 on the touch pad. No fuse, TVS or reverse protection (USB-C can't be reversed electrically). A deliberate cost-down.
Cable / charger	Not included (EU common-charger logic: every household already has USB-C). Optional €0.30 cable for gift packs.
On/off	No switch. The tree runs whenever plugged in; Pine naps on his own (and more in a dark room).

How the LED current works

The 240 LEDs are charlieplexed on 16 GPIO lines with one 100 Ω resistor per line. The firmware scans **by anode**: in phase *p* line *p* is driven high and the cathodes of the LEDs to be lit are driven low, 16 phases per frame. When *k* LEDs share one phase slot, they share the anode pin's current:

$$I_{LED}(k) = (V - V_f) / ((R + R_{drv}) \cdot (k + 1))$$ $V = 5.0$ V, $V_f \approx 2.75$ V, $R = 100$ Ω, $R_{drv} \approx 30$ Ω

LEDs lit in the slot (k)	Per LED	Anode pin total
1	8.7 mA	8.7 mA
4	3.5 mA	13.8 mA
8	1.9 mA	15.4 mA
15 (all)	1.1 mA	**16.2 mA**, which is under the 20 mA PY32F003 pin rating

Only one phase is active at a time, so **LED supply current can never exceed ≈ 16.2 mA**, whatever the animation. The simulator's "Raw scan" mode shows this sag, and its "Compensated" mode shows the firmware correction.

Which LEDs share a phase

Which anode/cathode pair sits at which position is no longer a fixed interleaved pattern: `hardware/scripts/optimize_leds.py` picks it so the board routes on two layers (see the README). The price is that neighbouring LEDs sometimes share an anode line: 17 % of the LED pairs closer than 3.2 mm do (6 % with the old `anode = n mod 16`). So when a compact blob is lit, the busiest phase carries more LEDs and each of them gets less current. Average over random blob positions, with the per-LED current from the formula above:

Lit disc, radius	LEDs lit	Busiest phase k, old → now	Current per LED in it, old → now
3 mm	8	1.8 → 2.7	6.3 → 4.7 mA
5 mm	18	2.7 → 4.1	4.7 → 3.4 mA
8 mm	35	3.7 → 6.0	3.7 → 2.5 mA
12 mm	68	6.0 → 8.8	2.5 → 1.8 mA

The pin current limit is unaffected (still ≤ 16.2 mA, whatever the pattern). What changes is how unevenly a compact pattern's brightness is spread over the scan phases, which is what the "Compensated" mode is for: the simulator uses the real map, so its Raw/Compensated views already show the new behaviour.

Consumption

Per-animation figures come from the simulator's electrical model: 8-bit BCM, gamma 2.2, 100 % brightness, **20 s average** after a 5 s warm-up, sampled 10× per second. The MCU at 24 MHz is included as ≈ 3 mA (the datasheet gives 1.5 mA at 24 MHz with peripherals off; the rest is a margin for the timer, ADC and scan interrupts).

Animation	Average supply	Max supply	Average power @ 5 V	Peak pin current
Fireflies	3.4 mA	4.1 mA	17 mW	14.4 mA
Fireworks	3.5 mA	5.5 mA	18 mW	13.8 mA
Rising sparks	3.6 mA	4.0 mA	18 mW	15.9 mA
Snowfall	5.6 mA	7.1 mA	28 mW	15.4 mA
Star ripple	5.8 mA	6.8 mA	29 mW	15.1 mA
Ornaments	6.4 mA	7.7 mA	32 mW	15.1 mA
Heartbeat	6.9 mA	14.4 mA	34 mW	16.0 mA
Spiral garland	7.2 mA	8.8 mA	36 mW	15.6 mA
Candle flicker	8.6 mA	10.0 mA	43 mW	16.2 mA
Soft radar	8.7 mA	15.8 mA	44 mW	16.2 mA
Fill & drain	8.9 mA	12.8 mA	44 mW	16.2 mA
Tinsel shimmer	9.0 mA	11.9 mA	45 mW	16.2 mA
Breathe	10.6 mA	18.6 mA	53 mW	16.2 mA
Twinkle	11.4 mA	13.1 mA	57 mW	15.7 mA
Northern lights	11.8 mA	17.2 mA	59 mW	16.2 mA
Mean of all 15 shows	**7.4 mA**		**37 mW**	
Theoretical maximum (all 240 LEDs at 100 %)		**19.2 mA**	**96 mW**	16.2 mA
Pine asleep (sleep face + dim breathing glow, estimate)	≈ 4 mA		≈ 20 mW	—

Energy per season: 37 mW × 8 h/day × 45 days ≈ **13 Wh**, about €0.01 of electricity. The USB charger's own no-load loss is larger than the tree.

Light output: XL-1608WWC-06 is 780 mcd at 20 mA, about 2.4 lm per LED for a Lambertian emitter. With LED current capped at 16.2 mA, the whole tree emits **≈ 2 lm**. That's roughly one Christmas-light bulb spread over 240 pixels, and it keeps the product far below the 60 lm lower limit of the EU light-source ecodesign rules (see eu-compliance).

Thermal: 0.1 W on a 40 cm^2 board, so no measurable temperature rise.

Known limitation: power banks

Many power banks switch off when the load stays below ~50–100 mA for 10–30 s. PixelPine draws 3–19 mA (≈ 7 mA on average), so on those banks it goes dark after a few seconds. The cheap fix (**not fitted in rev A**) is an N-MOSFET (AO3400, C20917, JLC basic, ≈ $0.02) with a 47 Ω resistor. It would pulse 100 mA for 100 ms every 8 s. The only free pin is NRST (PF2), so this variant would have to program via the UART bootloader instead of connect-under-reset. Average cost: +1.25 mA, +6 mW. Recommended only if customer feedback asks for it; the leaflet says "use a USB charger or computer port".

User interaction: Pine and the touch star (no radio, no app)

The tree has a personality, **Pine**. His job is to show light shows, but he loses focus, daydreams, gets sleepy and has moods (full design in animations.md). You interact with him only through the star:

Gesture	How it's detected	Typical reaction
Hover (finger ≤ ~2 cm)	small capacitance rise, ≥ 0.3 s	Pine looks at you and smiles, peeks out of a show
Tap	light touch < 0.45 s	next show · "Oh right, my job!" · a sleeping Pine stirs
Pet	light touch held > 0.6 s	content face, mood up, annoyance melts
Press	firm touch (flattened fingertip, bigger area)	insists: wakes him, next show, "HEY!" if grumpy
Squeeze	pinch front + back of the star (both faces are one electrode)	hug: blush and sparkles, or a squirm if grumpy
Tickle	3 taps within 1.4 s	giggle fit; too much makes him annoyed
Room lights off	LEDs used as light sensors between frames	he gets sleepy and naps in the dark
Power-up	—	"Good morning" yawn; remembers your **bond** (stored in flash)

Touch sensing. The star is a single electrode on both copper layers, joined by a plated hole, wired through R19 (1 kΩ, ESD) to **PA7, an ADC input**. The firmware measures its capacitance by charge sharing with the ADC's sample capacitor, which resolves fractions of a pF. That is enough to separate hover, touch, firm press and a two-sided pinch. A slow baseline tracker removes drift (humidity, temperature). Sampling happens between scan phases, so the LED matrix doesn't disturb it. No extra parts; the moved pin is the only hardware change.

Connectivity

Rev A has none, on purpose. No radio means:

- no Radio Equipment Directive (RED) and its cybersecurity delegated act, saving **€3–5 k** of extra testing,
- no USB-C data lines, so no USB compliance concerns and a cheaper 6-pin connector,
- no pairing, apps or firmware-update support burden.

Programming is done once in production through five bare pads on the back: 5 V, GND, SWDIO, SWCLK and NRST. A pogo-pin jig and any Arm SWD probe (CMSIS-DAP, J-Link, or a WCH-LinkE in Arm mode, ~$3) take ~5 s per board with *connect under reset*. While NRST is held low, PA13/PA14 are still in their SWD role. After boot the firmware takes them over as LED lines L8/L9, and PF4/BOOT0 as L15. Hackers can re-flash their own animations with the same pads.

Future variants (evaluated, not in rev A)

Variant	Extra parts	+ Cost	+ Compliance	What it enables
USB "Link"	16-pin USB-C, MCU → a USB-device MCU (e.g. CH32X035)	≈ $0.20	USB-IF optional; EMC unchanged	Firmware update by drag-and-drop, live control from the PC simulator
IR remote	IR receiver (needs one GPIO, so drop to 15 lines / 210 LEDs)	≈ $0.06 + remote $0.40	none	Change shows from the sofa
Music-reactive	MEMS mic + op-amp or PDM mic	≈ $0.15	none	Beat-synced animations
Bluetooth LE	CH582F / PY32 + BLE module instead of PY32F003	≈ $0.9	**RED + EN 300 328 + EN 18031 cybersecurity**, €3–5 k	Phone app, synchronised trees